Օբյեկտի փոխանցումը ֆունկցիային

1. Ինչու ենք օբյեկտ փոխանցում ֆունկցիային

Երբ ծրագիրը մեծանում է, նույն օբյեկտի հետ տարբեր գործողություններ պետք է կատարեն առանձին ֆունկցիաներ: Օրինակ՝ բանկային ավանդների պորտֆելը կարելի է ցուցադրել, վերլուծել կամ փոխել: Կարևոր հարցն այն է՝ ֆունկցիան աշխատո՞ւմ է սկզբնական օբյեկտի, թե՞ նրա առանձին պատճենի հետ:

Պատկերացնենք DepositPortfolio դասի օբյեկտ, որը պահում է {1000, 2000, 3000} ավանդները և p=10 տարեկան տոկոսադրույքը: Ֆունկցիաների կանչի ժամանակ կարող ենք ընտրել երեք եղանակ:

2. Երեք փոխանցման եղանակների համեմատություն

Ստորագրություն	Պատճեն	Կարո՞ղ է փոխել սկզբնականը
void f(Portfolio x)	Այո	Ոչ
void f(Portfolio& x)	Ոչ	Այո
void f(const Portfolio& x)	Ոչ	Ոչ

2.1. Արժեքով փոխանցում՝ T object

Ֆունկցիան ստանում է առանձին պարամետր-օբյեկտ: Այն սկզբնավորվում է կանչի պահին փոխանցված օբյեկտից: Սովորաբար այս պահին կանչվում է պատճենող կոնստրուկտորը: Պարամետրի փոփոխությունը չի փոխում սկզբնական օբյեկտը:

```
class Car {
public:
    int speed;
    Car(int s) : speed(s) {}
};
void changeCopy(Car car) { car.speed = 150; }
int main() {
    Car a(80);
    changeCopy(a);
    std::cout << a.speed; // 80
}
```

Քայլերով՝ a.speed-ը սկզբում 80 է, car-ը ստանում է 80 արժեքով պատճեն, փոխվում է միայն car.speed-ը, իսկ ֆունկցիայից դուրս a.speed-ը մնում է 80:

2.2. Հղումով փոխանցում՝ T& object

Հղումը նույն օբյեկտի այլ անունն է: Նոր օբյեկտ չի ստեղծվում, հետևաբար փոփոխությունը երևում է նաև main()-ում:

```
void changeOriginal(Car& car) { car.speed = 150; }
int main() {
    Car a(80);
    changeOriginal(a);
    std::cout << a.speed; // 150
}
```

Այս եղանակը նպատակահարմար է այն ֆունկցիաներում, որոնք դիտավորյալ փոխում են օբյեկտը՝ օրինակ set(), reverse() կամ sort() կանչելով:

2.3. Հաստատուն հղումով փոխանցում՝ const T& object

Ֆունկցիան աշխատում է նույն օբյեկտի հետ՝ առանց պատճենելու, բայց չի կարող փոփոխել այն ոչ-const մեթոդներով: Սա հատկապես հարմար է ընթերցման, հաշվարկի և ցուցադրման համար:

```
void show(const Car& car) {
    std::cout << car.speed;
    // car.speed = 150; // ՍԽԱԼ՝ const օբյեկտ
}
```

const-ը պաշտպանում է ֆունկցիայի միջոցով չնախատեսված փոփոխումից: Այն չի նշանակում, թե սկզբնական օբյեկտը այլևս երբեք չի կարող փոխվել:

3. Պատճենող կոնստրուկտոր. երբ և ինչու

Պատճենող կոնստրուկտորը ստեղծում է նոր օբյեկտ՝ արդեն գոյություն ունեցող օբյեկտի հիման վրա: Սովորական ձևն է ClassName(const ClassName& other): Հղումը թույլ է տալիս չստեղծել անվերջ պատճեններ, իսկ const-ը երաշխավորում է, որ աղբյուր օբյեկտը չի փոփոխվի:

```
class Car {
public:
    int speed;
    Car(int s) : speed(s) {}
    Car(const Car& other) : speed(other.speed) {
        std::cout << "Copy constructor\n";
    }
};
void inspect(Car x) { std::cout << x.speed << '\n'; }
int main() {
    Car a(80);
    Car b = a;   // պատճենող կոնստրուկտոր
    inspect(a);  // արժեքով պարամետրի պատճեն
}
```

Տարբերակենք. Car b = a; արտահայտությունը նոր b օբյեկտի սկզբնավորումն է, իսկ b = a; արտահայտությունը արդեն գոյություն ունեցող b-ին վերագրում է և սովորաբար կանչում է պատճենման վերագրման օպերատորը:

std::vector<double> դաշտի դեպքում ստանդարտ պատճենումը պատճենում է վեկտորի տարրերը, և երկու օբյեկտներն ունեն առանձին վեկտորներ: Հետևաբար այս լաբորատոր աշխատանքի համար new/delete և ձեռքով գրված պատճենող կոնստրուկտոր պարտադիր չեն:

4. Private դաշտեր և արտաքին ֆունկցիաներ

Արտաքին սովորական ֆունկցիան չի կարող ուղղակիորեն դիմել դասի private դաշտերին: Դրա համար դասը տալիս է public մեթոդներ՝ size(), get(), rate(), set(), print(): friend կիրառել պետք չէ:

```
class Car {
private:
```

```
    int speed;
public:
    Car(int s) : speed(s) {}
    int getSpeed() const { return speed; }
    bool setSpeed(int s) {
        if (s < 0) return false;
        speed = s;
        return true;
    }
};
void printCar(const Car& car) {
    std::cout << car.getSpeed(); // ճիշտ
    // std::cout << car.speed;    // սխալ՝ private
}
```

const մեթոդը գրվում է փակագծերից հետո՝ int getSpeed() const: Այդպիսի մեթոդը կարելի է կանչել const հղումով ստացված օբյեկտի վրա: Իսկ setSpeed() փոփոխող մեթոդը const օբյեկտի վրա կանչել չի թույլատրվում:

5. Գործնական ամբողջական օրինակ՝ DepositPortfolio

Խնդիր. Պահել ավանդների գումարները և տարեկան պարզ տոկոսադրույքը: Հաշվել տարեկան ընդհանուր եկամուտը, գտնել ամենամեծ վերջնական գումարով ավանդի ինդեքսը և հաշվել միջինից մեծ ավանդների քանակը: Ավելացնել երեք արտաքին ֆունկցիա՝ ընթերցող, պատճենը փոխող և սկզբնականը փոխող:

5.1. Հաշվարկների բանաձևեր

Տարեկան եկամուտը յուրաքանչյուր ավանդի համար՝ amount × p / 100: Ընդհանուր եկամուտը՝ այդ արժեքների գումարը: Վերջնական գումարը՝ amount × (1 + p/100): Միջինը՝ բոլոր ավանդների գումարը բաժանած դրանց քանակին:

Օրինակ՝ [1000, 2000, 3000], p=10: Եկամուտները՝ 100, 200, 300, ընդհանուր՝ 600: Վերջնական գումարները՝ 1100, 2200, 3300, լավագույն ինդեքսը՝ 2 (ինդեքսավորումը սկսվում է 0-ից): Միջինը 2000 է, դրանից մեծ է միայն 3000-ը՝ քանակը 1:

5.2. Ամբողջական աշխատող C++ ծրագիր

```
#include <iostream>
#include <vector>
#include <cmath>
#include <stdexcept>
using namespace std;

class DepositPortfolio {
private:
    vector<double> data;
    double p;
    static bool valid(double x) { return isfinite(x) && x > 0; }
public:
    DepositPortfolio(const vector<double>& amounts, double rate)
        : data(amounts), p(rate) {
        if (data.size() < 3 || data.size() > 30 ||
            !isfinite(p) || p < 0 || p > 100)
            throw invalid_argument("Invalid size or rate");
        for (double x : data)
```

```
            if (!valid(x)) throw invalid_argument("Invalid amount");
    }
    int size() const { return static_cast<int>(data.size()); }
    double get(int i) const { return data.at(i); }
    double rate() const { return p; }
    bool set(int i, double value) {
        if (i < 0 || i >= size() || !valid(value)) return false;
        data[i] = value;
        return true;
    }
    void print() const {
        cout << "Rate: " << p << "% | Amounts:";
        for (double x : data) cout << ' ' << x;
        cout << '\n';
    }
    double annualIncome() const {
        double sum = 0;
        for (double x : data) sum += x * p / 100.0;
        return sum;
    }
    int bestIndex() const {
        int best = 0;
        for (int i = 1; i < size(); ++i)
            if (data[i] * (1 + p / 100.0) >
                data[best] * (1 + p / 100.0) + 1e-9)
                best = i;
        return best;
    }
    int countAboveAverage() const {
        double sum = 0;
        for (double x : data) sum += x;
        double avg = sum / size();
        int count = 0;
        for (double x : data)
            if (x > avg + 1e-9) ++count;
        return count;
    }
};

void analyze(const DepositPortfolio& object) {
    object.print();
    cout << "Annual income: " << object.annualIncome() << '\n';
    cout << "Best index: " << object.bestIndex() << '\n';
    cout << "Above average: " << object.countAboveAverage() << '\n';
}
void changeCopy(DepositPortfolio object, int index, double value) {
    if (!object.set(index, value)) cout << "Invalid change\n";
    object.print();
}
void changeOriginal(DepositPortfolio& object, int index, double value) {
    if (!object.set(index, value)) cout << "Invalid change\n";
}

int main() {
    DepositPortfolio a({1000, 2000, 3000}, 10);
    analyze(a);
    changeCopy(a, 0, 9000);
    cout << "After copy: "; a.print();
    changeOriginal(a, 0, 9000);
    cout << "After reference: "; a.print();
```

```
}
```

Ուշադրություն. օրինակում ընդունված են դրական ավանդներ և 0–100% տոկոսադրույք: Քանի որ առաջադրանքի մեջ p-ի թույլատրելի միջակայքը հստակ տրված չէ, դասավանդողը կարող է այն փոխել ըստ իր տարբերակի կանոնների:

5.3. Ծրագրի քայլ առ քայլ բացատրություն

• Կոնստրուկտորը ստուգում է սկզբնական չափը, տոկոսադրույքը և յուրաքանչյուր ավանդը: Սխալ տվյալների դեպքում օբյեկտ չի ստեղծվում:

• analyze(const DepositPortfolio&) ստանում է ընթերցման հասանելիություն՝ առանց պատճենի և կանչում է միայն const մեթոդները:

• changeCopy(DepositPortfolio object, ...) ստանում է առանձին վեկտոր ունեցող պատճեն: set()-ը փոխում է միայն այդ պատճենը:

• changeOriginal(DepositPortfolio& object, ...) աշխատում է սկզբնական a օբյեկտի հետ:

• set()-ը սխալ ինդեքսի կամ արժեքի դեպքում վերադարձնում է false և չի փոխում նախկին տվյալները:

• bestIndex()-ը հավասար լավագույն արդյունքների դեպքում պահում է առաջին ինդեքսը, քանի որ փոխում է միայն խիստ մեծ լինելու դեպքում:

6. Ինչպե՞ս վերադարձնել նոր օբյեկտ

Երբ պետք է ստանալ նոր, փոփոխված հավաքածու՝ առանց սկզբնականը փոխելու, ֆունկցիան կարող է ընդունել const հղում, ստեղծել պատճեն և վերադարձնել այն արժեքով: Ժամանակակից C++-ում վերադարձի պատճենումը հաճախ օպտիմիզացվում է, բայց օպտիմիզացման մանրամասները չպետք է շփոթել փոխանցման տրամաբանության հետ:

```
DepositPortfolio modified(const DepositPortfolio& source) {
    DepositPortfolio result(source); // vector-ը նույնպես պատճենվում է
    result.set(0, 5000);
    return result;
}
// Օգտագործում՝ DepositPortfolio b = modified(a);
// a-ն անփոփոխ է, b-ն՝ նոր օբյեկտ:
```